

Antecedentes y Contexto del Proyecto

La Computación Afectiva En el desarrollo de software moderno y el análisis de datos, cada vez es más importante comprender no solo qué hacen los usuarios, sino cómo se sienten. Esta disciplina, conocida como Computación Afectiva, busca que los sistemas puedan detectar e interpretar el estado emocional de las personas para ofrecer mejores experiencias.

El Modelo de las Emociones Para que un programa informático (como el que vas a construir en Python) pueda "entender" una emoción, necesitamos traducir los sentimientos a números. En este taller utilizaremos una versión simplificada del **Modelo Circunflejo de Russell**. Este modelo propone que todas las emociones humanas pueden medirse combinando dos variables continuas:

- **Valencia:** Define qué tan positiva o negativa es una experiencia (desde -1.0 para muy desagradable, hasta +1.0 para muy agradable).
- **Activación (Arousal):** Define el nivel de energía física o mental (desde -1.0 para muy relajado o somnoliento, hasta +1.0 para muy alerta o tenso).

Tu Misión Los datos sin procesar son solo números. Un registro con Valencia de 0.8 y Activación de 0.9 no significa nada por sí solo. Tu misión como desarrollador es construir la lógica matemática y estructural en Python que tome estos datos masivos (extraer datos de usuarios desde una base de datos relacional y combinarlos con un registro de respuestas afectivas almacenado en un archivo CSV), los limpie, cruce los valores en un plano bidimensional y le asigne una etiqueta comprensible para los humanos (como "Entusiasta" o "Frustrado"). Al finalizar, habrás construido el motor lógico de un sistema de análisis de experiencia de usuario.

Modelo Circumplejo de Russell



1. Requisitos Técnicos Obligatorios

El código entregado debe integrar de manera obligatoria los siguientes componentes:

- **Codificación de Funciones (Modularización):** El código no puede ser un script plano. Se debe diseñar un sistema modular con funciones específicas que tengan responsabilidades únicas (ej. una función para conectar, otra para calcular, otra para clasificar). Cada función debe incluir sus respectivos parámetros y retorno de datos.
- **Manejo de Excepciones (`try-except-finally`):** El programa debe ser tolerante a fallos. Se deben anticipar y capturar errores críticos como la ausencia del archivo CSV, fallas en la conexión de la base de datos, inconsistencias en los tipos de datos (por ejemplo, intentar convertir un texto vacío a flotante) o divisiones por cero al calcular promedios.
- **Estructuras de Control:** Uso riguroso de bucles (`for/while`) para recorrer los registros y estructuras condicionales anidadas o compuestas (`if-elif-else`) para la toma de decisiones algorítmicas.
- **Estructuras de Datos Comunes:** Manipulación de colecciones nativas de Python (listas, diccionarios y tuplas) para organizar, filtrar y agrupar la información en memoria antes de generar el reporte final.